UCSH1-301

ASSIGNMENT

Topic : Hardware Based Control Unit

Name: Kapakayala Siva Sai Pavan

Regd No.: 234219

Hardware-based Control Unit Design...

Control Unit Operation:

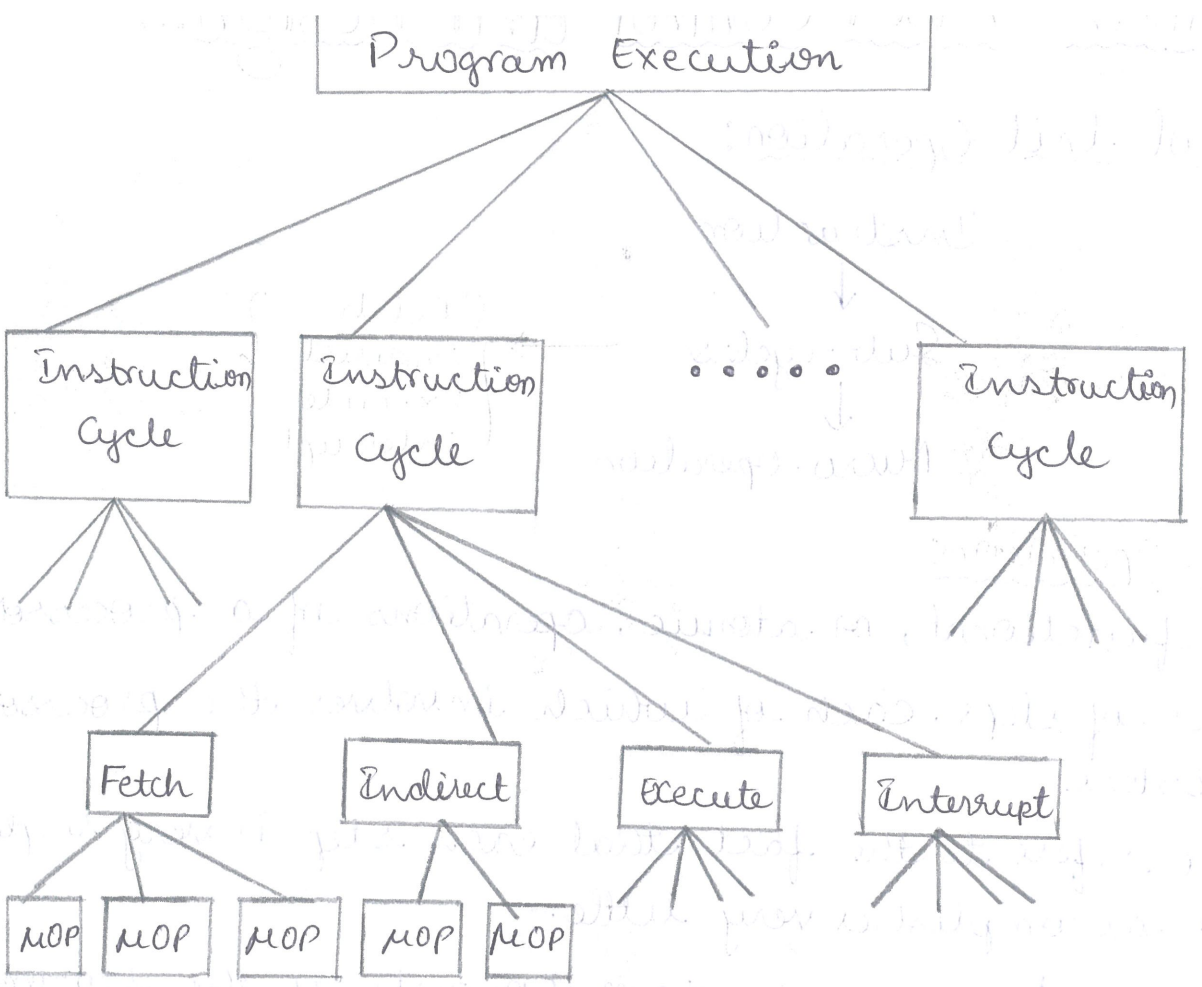
Instruction
↓
Sub-cycles → { Fetch, Indirect, Execute, Interrupt }
↓
Micro-operation

Micro-Operations:

* The functional, or atomic, operations of a processor.
* Series of steps, each of which involves the processor registers.
* Micro refers to the fact that each step is very simple and accomplishes very little.
* The execution of a program consists of the sequential execution of instructions.
  - Each instruction is executed during an instruction cycle made up of shorter subcycles. (fetch, indirect, execute, interrupt)
  - The execution of each subcycle involves one or more shorter operations. (micro-operations).

Program Execution:

* Instruction cycle
  * Fetch
  * Indirect
  * Execute
  * Interrupt
  * Micro-operation.
* Micro-operation.



Constituent Elements of a Program Execution.

Fetch Cycle:

t_1 : MAR ← PC

MAR ⇒ SB, LOAD, READ

{ PC, MAR, MBR, IR } — System Bus (SB)

* MAR is connected to system bus.
* MAR contents is now loaded into address bus.
* Read command is issued by the control bus.

t_2 : MBR ← Memory

t_3 : IR ← MBR

The MBR can be reused in the subsequent subcycle (indirect) for storing the operand inside the MBR.

* Occurs at the beginning of each instruction cycle and causes an instruction to be fetched from memory.
* Four registers are involved:
  * Memory Address Register (MAR)
    - Connected to address bus.
    - Specifies address for read or write operation.
  * Memory Buffer Register (MBR)
    - Connected to data bus
    - Holds data to write or last data read.
  * Program Counter (PC)
    - Holds address of next instruction to be fetched.
  * Instruction Register (IR)
    - Holds last instruction fetched.

MAR	
MBR	
PC	0000000001100100
IR	
AC	

(a) Beginning (before step1)

MAR	0000000001100100
MBR	
PC	0000000001100100
IR	
AC	

(b) After first step

MAR	0000000001100100
MBR	0001000000100000
PC	0000000001100100
IR	
AC	

(c) After second step

MAR	0000000001100100
MBR	0001000000100000
PC	0000000001100100
IR	0001000000100000
AC	

(d) After third step.

Time	Fetch Cycle
t_1	Move the address from the Program Counter (PC) to the Memory Address Register (MAR)
t_2	* Place the address in the MAR onto the address bus. * Control unit issues a READ command on the control bus. * The instruction from memory at the specified address is placed on the data bus and copied into the Memory Buffer Register (MBR).
t_3	* Transfer the instruction from the MBR to the Instruction Register (IR) * This frees the MBR for any potential indirect operations.

t_1 : MAR $\leftarrow$ (PC)

t_2 : MBR $\leftarrow$ Memory

PC $\leftarrow$ (PC) + I

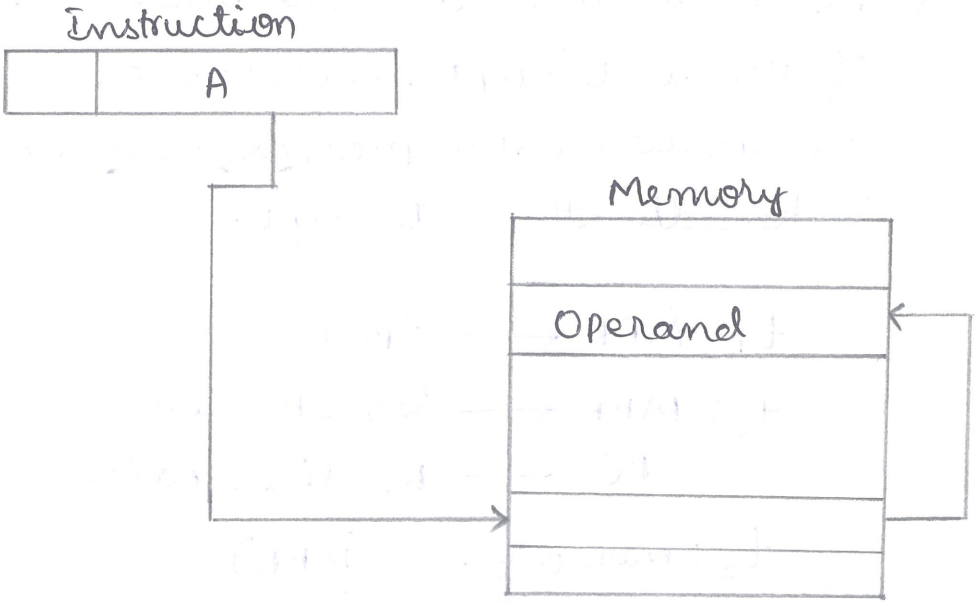
t_3 : IR $\leftarrow$ (MBR)

Time	Indirect Cycle with Indirect Addressing Mode (A)
t_1	The address field of the instruction in the Instruction Register (IR) is transferred to the Memory Address Register (MAR)
t_2	The address in the MAR is used to fetch the actual operand address from memory, which is stored in the MBR.
t_3	* Transfer the instruction from the MBR to the Instruction Register (IR). * This frees the MBR for any potential indirect operations.

t_1: MAR $\longleftarrow$ (IR (Address))

t_2: MBR $\longleftarrow$ Memory

t_3: IR (Address) $\longleftarrow$ (MBR (Address))



Time	Interrupt Cycle
t_1	* The Contents of the Program Counter (PC) are transferred to the Memory Buffer Register (MBR). * This saves the current instruction address for returning after the interrupt.
t_2	* The Memory Address Register (MAR) is loaded with the address where the PC's Contents will be saved. (A location in Main memory). * The PC is then loaded with the address of the start of the interrupt processing routine.
t_3	* The PC is then loaded with the address of the interrupt. processing routine. This is where the processor will jump to handle the interrupt.

t_1: MBR $\longleftarrow$ (PC)

t_2: MAR $\longleftarrow$ Save_Address

PC $\longleftarrow$ Routine_Address

t_3: Memory $\longleftarrow$ (MBR)

Execute Cycle:

* Because of the variety of opcodes, there are number of different sequences of micro-operations that can occur.

* Instruction decoding:
  - The control unit examines the opcode and generates a sequence of micro-operations based on the value of the opcode.

* A simplified add instruction:
  * ADD R_1, X (which adds the contents of the location X to Register R_1)
    - In the first step the address portion of the IR is loaded into the MAR.
    - Then the referenced memory location is read.
    - Finally the contents of R_1 and MBR are added by the ALU.
    - Additional micro-operations may be required to extract the register reference from the IR and perhaps to stage the ALU inputs or outputs in some intermediate registers.

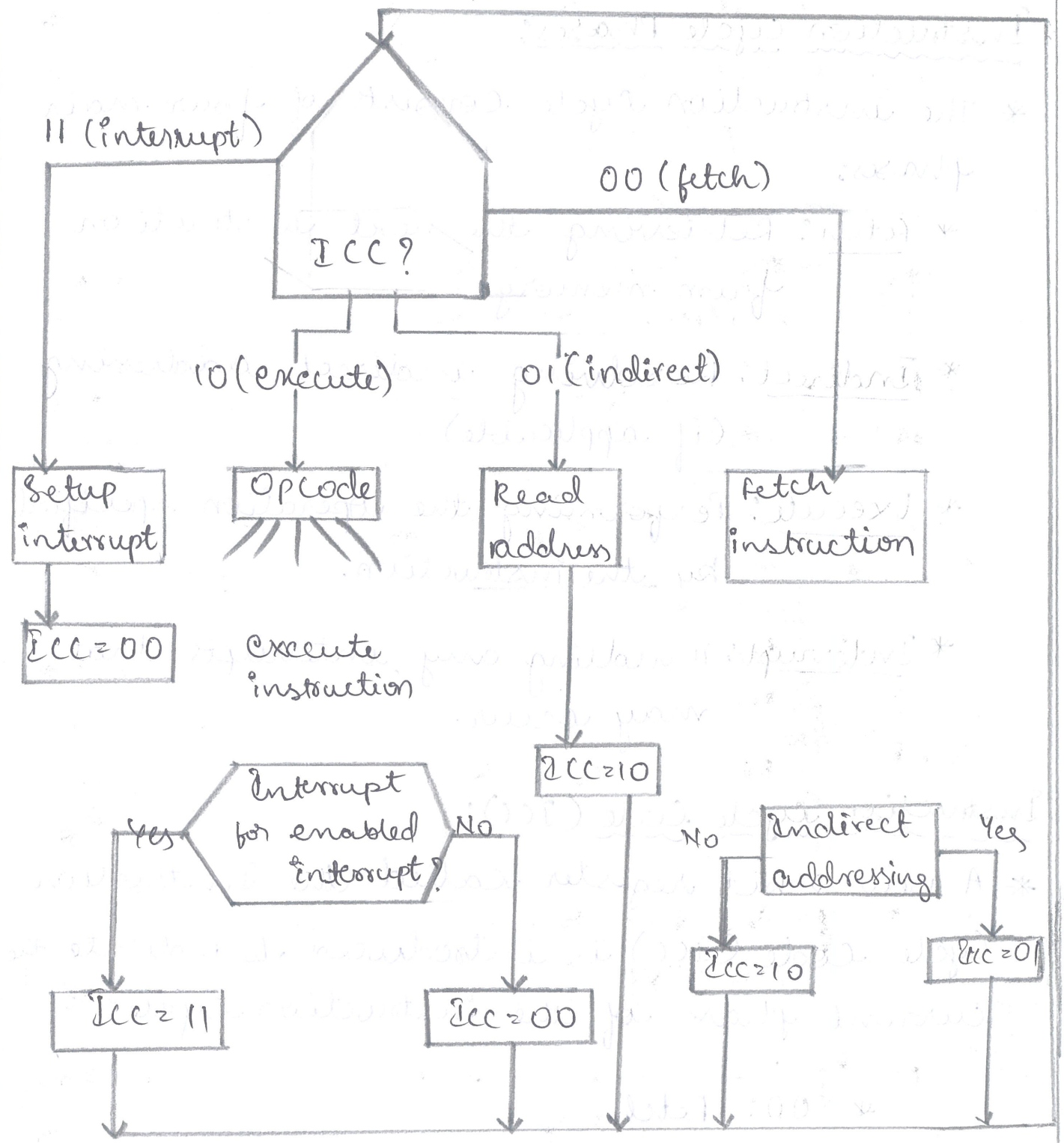
t_1: MAR ← IR(Address)

t_2: MBR ← Memory

t_3: R_1 ← MBR + (R_1)

Rules for Micro-Operations Grouping:

* Proper sequence must be followed.
  - MAR ← (PC) must precede MBR ← (Memory)
* Conflicts must be avoided
  - Must not read and write same register at same time.
  - MBR ← (memory) and IR ← (MBR) must not be in same cycle.
* One of the micro-operations involves an addition
  - To avoid duplication of circuiting, this addition could be performed by the ALU.
  - The use of the ALU may involve additional micro-operations, depending on the functionality of the ALU and the organization of the processor.



Flow Chart for Instruction cycle

Instruction Cycle Phases:

* The instruction cycle consists of four main phases:
  * Fetch: Retrieving the next instruction from memory.
  * Indirect: Resolving indirect addressing (if applicable)
  * Execute: Performing the operation specified by the instruction.
  * Interrupt: Handling any interrupts that may occur.

Instruction Cycle Code (ICC):

* A new 2-bit register called the Instruction Cycle Code (ICC) is introduced to indicate the current phase of the instruction cycle:
  * 00: Fetch
  * 01: Indirect
  * 10: Execute
  * 11: Interrupt

* At the end of each cycle, the ICC is updated to reflect the next state.

Flow of Cycles:

* The indirect cycle is always followed by the execute cycle.
* The interrupt cycle is always followed by the fetch cycle.
* For both the fetch and execute cycles, the next phase depends on the current system state.

Micro-Operations:

* Each phase involves specific sequences of micro-operations that define how the processor interacts with its registers, ALU and memory.

Control Unit Role:

* The Control unit orchestrates the execution of these micro-operations based on the state of the ICC and the instruction sequence.
* The Control flow is more complex in actual processors but allows a similar structure, emphasizing that processor operation is essentially a sequence of micro-operations.

Control Unit (HCU) and Microprogrammed Control Unit (MCU)?

Feature	HCU	MCU
Implementation	Fixed logic circuits eg- decoders and multiplexer	Control memory with microinstructions
Control word Generation	Directly from combinational logic	Stored in control memory as micro instructions.
Control memory	Not used; uses fixed logic circuits	Uses control memory to store microinstructions.
Data Path	Fixed pathways, less adaptable	Flexible datapaths; Can adapt easily.
Speed	Generally faster	Generally slower due to memory access
Flexibility	less flexible; hard to modify	more flexible; easy to modify
Complexity	Increase with instruction set complexity.	Can manage complexity through organized microinstructions.
Cost	Typically cheaper in simple designs	Potentially more expensive due to memory.

Design change	Requires hardware design	Only requires changes to micro-program
Micro-operation control	Directly from combinat-ional logic	From stored micro-instructions.
Control word example	11010000 fixed for an operation	11000000, 00010000 etc (varies per microinstruction.
Execution flexibility	limited; hard to modify control logic	High; can easily change control words by updating microinstructions.